EXVUL

SMART CONTRACT AUDIT REPORT

Hedgehood Vaults Smart Contract

OCTOBER 2026

Contents

1. EXECUTIVE SUMMARY **3**
1.1 Methodology . . . 3

2. FINDINGS OVERVIEW **7**
2.1 Project Info And Contract Address . . . 7
2.2 Summary . . . 7
2.3 Key Findings . . . 8

3. DETAILED DESCRIPTION OF FINDINGS **9**
3.1 Force settlement bypasses the oracle status check . . . 9
3.2 Settlement bypasses the oracle status check . . . 11
3.3 One LP invalidates other providers' rotation quotes . . . 13
3.4 Credit order selects the depositor who receives rounding residuals . . . 15
3.5 A negative rebase makes basket redemption depend on order . . . 17
3.6 An empty revert aborts every later bid fill . . . 19

4. CONCLUSION **21**

5. APPENDIX **22**
5.1 Basic Coding Assessment . . . 22
5.2 Advanced Code Scrutiny . . . 25

6. DISCLAIMER **27**

7. REFERENCES **28**

8. About Exvul Security **29**

1. EXECUTIVE SUMMARY

ExVul Web3 Security reviewed the **Hedgehood Vaults** core OptionVault v2 smart contracts at the `audit-v2-freeze-2026-10-21-r2` source freeze. The assessment covered the 16 client-designated production implementations under `contracts/src/v2/`, comprising approximately 2,063 nonblank, non-comment Solidity lines.

The review used five analysis passes covering settlement and oracle boundaries, epoch and share accounting, quote and bid authorization, rotation execution, basket behavior, access control, and cross-contract state transitions. The repository baseline of 78 `test/v2/*` tests passed with Solidity 0.8.26. Those baseline tests confirm build health but do not provide regression coverage for the new findings. Interfaces, tests, deployment scripts, shared v1 modules, share-collateral code, third-party dependencies, and off-chain services were excluded from this report's core scope.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

IMPACT

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Hedgehood Vaults	28/10/2026 - 28/10/2026	Solidity

Repository

https://github.com/0xHedgeHood/hedgehood-vaults

Commit Hash

`698276a627972a50822d18b21239203a6ec286e2`

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	2
MEDIUM	1
LOW	1
INFO	2



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Force settlement bypasses the oracle status check	Acknowledge
HIGH	Settlement bypasses the oracle status check	Acknowledge
MEDIUM	One LP invalidates other providers' rotation quotes	Acknowledge
LOW	Credit order selects the depositor who receives rounding residuals	Acknowledge
INFO	A negative rebase makes basket redemption depend on order	Acknowledge
INFO	An empty revert aborts every later bid fill	Acknowledge

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Force settlement bypasses the oracle status check

SEVERITY: HIGH **STATUS:** Acknowledge

PATH:

`contracts/src/v2/EpochFund.sol`

DESCRIPTION:

After `FORCE_SETTLE_DELAY`, any address can call `EpochFund.forceSettle`. The function reads the public `finalizedPrice` mapping getter and sends that raw value to the engine's settlement hook. In contrast, the oracle's checked `settlementPrice` path verifies that the stock is not paused and that its current `uiMultiplier` matches the multiplier bound for the epoch. A corporate-action status change therefore makes the checked oracle accessor revert while `forceSettle` still consumes the stored price.

```
function forceSettle(address engine) external nonReentrant {
    Engine storage g = _openEngine(engine);
    uint256 epochId = currentEpoch;
    if (block.timestamp < uint256(epochSchedules[epochId].expiry) + FORCE_SETTLE_DELAY)
    revert TooEarly();
    uint256 price = oracle.finalizedPrice(engine, epochId);
    (bool ok, uint256 paidStock, uint256 paidUsdg, uint256 fee) =
    _callSettlementHook(engine, epochId, price);
    // ... books the returned payout when the result is within collateral and fee bounds
}
```

The mainnet v2 source is documented as byte-identical to the frozen v2 source. Reachability is source-confirmed, but the review did not establish that a corporate-action state change and late force settlement have occurred in production.

IMPACT:

Buyers can be credited from a settlement price that the oracle explicitly rejects after a stock pause or multiplier change. Depending on the option positions, this can redistribute locked stock or USDG between buyers and fund depositors. Engine collateral and fee caps bound the amount, but they do not restore the oracle's corporate-action invariant.

RECOMMENDATIONS:

Expose an oracle accessor that validates the status associated with an explicitly supplied vault or engine namespace, and use it in `forceSettle`. If the checked read fails, follow the existing zero-settlement fallback rather than passing a rejected price into the engine hook.

```
 interface IEpochFundOracle {
-    function finalizedPrice(address vault, uint256 epochId) external view returns (uint256);
+    function checkedFinalizedPrice(address vault, uint256 epochId) external view returns
    (uint256);
 }

-uint256 price = oracle.finalizedPrice(engine, epochId);
+uint256 price;
+try oracle.checkedFinalizedPrice(engine, epochId) returns (uint256 checkedPrice) {
+    price = checkedPrice;
+} catch {
+    _forceSettleZero(engine, g);
+    return;
+}
```

The oracle implementation of `checkedFinalizedPrice` should require a nonzero finalized price and call `_stockStatus(_contexts[vault][epochId].stockMultiplier)` before returning it.

3.2 Settlement bypasses the oracle status check

SEVERITY: HIGH **STATUS:** Acknowledge

PATH:

`contracts/src/v2/OptionBook.sol`

DESCRIPTION:

Anyone can call `OptionBook.settle` after expiry while the relevant epoch is locked. The function reads the oracle's raw public `finalizedPrice` mapping getter rather than `settlementPrice(epochId)`, which is the checked accessor for the calling book. The raw getter does not verify `oraclePaused` or ensure that the current stock `uiMultiplier` matches the value bound to the epoch.

```
function settle() external nonReentrant {
    uint256 epochId = fund.currentEpoch();
    if (fund.phase() != IEpochFundEngineHost.Phase.Locked || openEpoch != epochId) revert
    WrongPhase();
    uint256 price = oracle.finalizedPrice(address(this), epochId);
    (uint256 paidStock, uint256 paidUsdg, uint256 fees) = _settleBook(epochId, price, false);
    fund.settleEpoch(paidStock, paidUsdg, fees, price);
}
```

This directly bypasses the status check implemented by `BrokerVaultOracle.settlementPrice` for the same book and epoch.

IMPACT:

Following a pause or corporate-action multiplier change, any caller can finalize option payouts at a price that the oracle's checked API rejects. The resulting buyer payout can consume locked collateral and reduce fund NAV. Collateral bounds limit absolute loss, but the settlement can still violate the intended buyer/depositor allocation.

RECOMMENDATIONS:

Replace the raw mapping lookup with the checked, caller-namespaced accessor. Preserve the existing delayed out-of-the-money fallback only for the documented no-price case; do not treat a corporate-action rejection as a valid stored price.

```
interface IOptionBookOracle {
    function price() external view returns (uint256);
```

```
     function bindEpoch(uint256 epochId, uint64 expiry, uint256 stockMultiplier) external
    returns (bytes32);
-    function finalizedPrice(address vault, uint256 epochId) external view returns (uint256);
+    function settlementPrice(uint256 epochId) external view returns (uint256);
 }

-uint256 price = oracle.finalizedPrice(address(this), epochId);
+uint256 price = oracle.settlementPrice(epochId);
```

3.3 One LP invalidates other providers' rotation quotes

SEVERITY: MEDIUM **STATUS:** Acknowledge

PATH:

`contracts/src/v2/RfqRotator.sol`

DESCRIPTION:

Rotation nonces are stored only by vault even though multiple liquidity providers can be authorized for one vault. Every LP initially signs with the same `nonces[vault]` value. Executing any one valid quote increments that vault-global nonce and invalidates every outstanding quote from every other LP, including a quote with better terms that was signed independently.

```
mapping(address => mapping(address => bool)) public liquidityProvider;
mapping(address => uint256) public nonces;

function rotate(RotationQuote calldata q, bytes calldata signature) external nonReentrant {
    if (
        !registry.isVault(q.vault) || !liquidityProvider[q.vault][q.lp]
            || q.nonce != nonces[q.vault]
        // ...
    ) revert BadTerms();
    // ...
    ++nonces[q.vault];
    // ...
}
```

Because submission is permissionless, a searcher or LP can choose which equal-nonce quote lands first. The fund's on-chain price band limits the terms of the executed rotation, but does not preserve competition among authorized providers.

IMPACT:

An authorized LP or transaction-ordering actor can invalidate competing quotes and cause the vault to miss a better in-band execution. This is a repeatable execution-quality and liveness issue; direct principal theft outside the configured rotation band was not established.

RECOMMENDATIONS:

Track nonces per vault and LP so one provider's execution or cancellation does not mutate another provider's quote domain. If the protocol intentionally wants a vault-wide one-shot auction, represent it with a separate round identifier and deterministic quote-selection rules.

```
-mapping(address => uint256) public nonces;
+mapping(address => mapping(address => uint256)) public nonces;

-q.nonce != nonces[q.vault]
+q.nonce != nonces[q.vault][q.lp]

-++nonces[q.vault];
+++nonces[q.vault][q.lp];
```

3.4 Credit order selects the depositor who receives rounding residuals

SEVERITY: LOW **STATUS:** Acknowledge

PATH:

`contracts/src/v2/EpochFund.sol`

DESCRIPTION:

Frozen deposit shares are allocated against the mutable assets and shares left in the cohort. The final account receives the entire residual. Because `creditAccount(account)` is permissionless, any caller can choose the credit order and therefore influence which depositor becomes last.

```
if (a.depositStock == e.depositStockLeft && a.depositUsdg == e.depositUsdgLeft) {
    return (e.depositSharesLeft, 0, 0);
}
uint256 left = Math.mulDiv(e.depositStockLeft, e.navPrice, VALUE_SCALE) + e.depositUsdgLeft;
uint256 value = Math.mulDiv(a.depositStock, e.navPrice, VALUE_SCALE) + a.depositUsdg;
shares = Math.mulDiv(value, e.depositSharesLeft, left);
```

For example, with cohort value 3, two minted shares, and deposits worth 1 and 2, crediting the smaller account first produces allocations 0 and 2, while crediting the larger account first produces 1 and 1. This is a deterministic arithmetic illustration of the source behavior, not a production-state replay.

IMPACT:

A caller can direct rounding residuals to a chosen depositor. The absolute difference is bounded by integer-rounding dust per allocation, but the result is order-dependent and violates deterministic pro-rata share issuance.

RECOMMENDATIONS:

Calculate every depositor's allocation from the immutable cohort totals. After all accounts have been credited, burn any unallocated residual held by the fund or assign it using a deterministic policy that callers cannot influence.

```
-if (a.depositStock == e.depositStockLeft && a.depositUsdg == e.depositUsdgLeft) return
    (e.depositSharesLeft, 0, 0);
-uint256 left = Math.mulDiv(e.depositStockLeft, e.navPrice, VALUE_SCALE) + e.depositUsdgLeft;
-if (left == 0) return (0, 0, 0);
+uint256 total = Math.mulDiv(e.depositStock, e.navPrice, VALUE_SCALE) + e.depositUsdg;
```

```
+if (total == 0) return (0, 0, 0);
 uint256 value = Math.mulDiv(a.depositStock, e.navPrice, VALUE_SCALE) + a.depositUsdg;
-shares = Math.mulDiv(value, e.depositSharesLeft, left);
+shares = Math.mulDiv(value, e.depositShares, total);

 e.depositStockLeft -= m.depositStock;
 e.depositUsdgLeft -= m.depositUsdg;
 e.depositSharesLeft -= shares;
+if (e.depositStockLeft == 0 && e.depositUsdgLeft == 0 && e.depositSharesLeft != 0) {
+    _burn(address(this), e.depositSharesLeft);
+    e.depositSharesLeft = 0;
+}
```

3.5 A negative rebase makes basket redemption depend on order

SEVERITY: INFO **STATUS:** Acknowledge

PATH:

`contracts/src/v2/ShareBasket.sol`

DESCRIPTION:

`ShareBasket` can be deployed directly with any ERC-20 component. Redemption always transfers the immutable configured units instead of a pro-rata share of the component balance. If a directly configured component negatively rebases or otherwise reduces the basket's balance, early redeemers receive the original fixed amount and later equal holders receive less or revert.

```
function redeem(uint256 amount, address to) external nonReentrant {
    if (amount == 0) revert BadAmount();
    _burn(msg.sender, amount);
    uint256[] memory amounts = previewRedeem(amount);
    for (uint256 i; i < amounts.length; ++i) {
        if (amounts[i] != 0) _components[i].safeTransfer(to, amounts[i]);
    }
}
```

For two equal basket holders backed by 200 component units, a 50% negative rebase leaves 100 units. The first holder can redeem the configured 100 units, after which the second holder's equal redemption reverts. The official `BasketFactory` mitigates this scenario by accepting only registered `EpochFund` vault shares, which are non-rebasing under the reviewed implementation. The issue therefore applies to direct arbitrary-component deployments, not factory-created official baskets.

IMPACT:

Users of a directly deployed basket with a negative-rebasing or confiscatable component can face first-redeemer advantage and failed later redemptions. No exploit against factory-created official baskets was established.

RECOMMENDATIONS:

Either restrict supported components to tokens whose balances cannot decrease asynchronously, or cap each redemption at the holder's pro-rata share of the basket's current balance. Document how positive rebase surplus is handled.

```
 function redeem(uint256 amount, address to) external nonReentrant {
     if (amount == 0) revert BadAmount();
+    uint256 supply = totalSupply();
     _burn(msg.sender, amount);
     uint256[] memory amounts = previewRedeem(amount);
     for (uint256 i; i < amounts.length; ++i) {
+        amounts[i] = Math.min(
+            amounts[i],
+            Math.mulDiv(_components[i].balanceOf(address(this)), amount, supply)
+        );
         if (amounts[i] != 0) _components[i].safeTransfer(to, amounts[i]);
     }
 }
```

3.6 An empty revert aborts every later bid fill

SEVERITY: INFO **STATUS:** Acknowledge

PATH:

`contracts/src/v2/BidDesk.sol`

DESCRIPTION:

`BidDesk.fill` is designed to skip a bid that becomes invalid or unfunded while continuing with later bids. Its catch block instead interprets any zero-length revert payload as proof that the subcall exhausted gas and reverts the whole batch. A bidder controls its USDG allowance and can cause `fillOne` to fail at `safeTransferFrom`; if the deployed USDG failure path can reach the catch block with empty revert data, the bidder can place first in a batch, revoke allowance, and abort all later fills.

```
try this.fillOne(r, fills[i], feeBps, usdg) returns (uint8 slot, uint256 premium, uint256
    fee) {
    emit Filled(r.vault, r.epochId, b.buyer, r.book, slot, bidDigest(b), fills[i].size,
    premium, fee);
    ++filled;
} catch (bytes memory reason) {
    if (reason.length == 0) revert InsufficientGas();
    emit Skipped(r.vault, r.epochId, b.buyer, bidDigest(b), reason);
    lastReason = reason;
}
```

The Solidity control-flow primitive is confirmed, but the review did not verify that the production USDG implementation and pinned OpenZeppelin transfer wrapper can produce an empty revert for insufficient allowance. This item is therefore an unverified dependency-dependent lead, not a confirmed production exploit.

IMPACT:

If the dependency condition holds, one bidder can cause an operator's whole ordered batch to revert and delay every later fill. The effect is denial of service and execution delay; no direct asset loss was established. If all reachable token failures carry nonempty revert data, this path is not exploitable through allowance revocation.

RECOMMENDATIONS:

Test the exact deployed USDG bytecode and pinned transfer library for insufficient allowance, insufficient balance, pause, and blacklist failures. Do not infer out-of-gas solely from revert-data length. Instead, ensure token-call failures are normalized to an explicit nonempty custom error inside `fillOne`, and use an explicit per-fill gas budget plus a reserved-gas check if gas-exhaustion detection is required.

4. CONCLUSION

In this audit, we thoroughly analyzed the **Hedgehood Vaults** core smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption. https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation. https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure. https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems. https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors. https://cwe.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul